

DOKUMENTACE KOMPILÁTORU JAZYKA IFJ25

Tým xobrale00 - varianta VV_BVS
Rozsireni

Vojtěch Kadlec	xkadlev00	25 %
Jakub Kosinka	xkosinj00	25 %
Aleš Obr	xobrale00	25 %
Tomáš Zvoníček	xzvonit00	25 %

*Vedoucí týmu je v seznamu výše označen **tučně**

3. 12. 2025

Obsah

1. Úvod.....	2
2. Lexikální analýza.....	2
3. Syntaktická a sémantická analýza	3
3.1 Syntaktická analýza	3
3.2 Sémantická analýza	3
4. Generátor strojového kódu	4
5. Použité algoritmy a datové struktury	5
5.1 Tabulka symbolů	5
5.2 AST	5
6. Použité nástroje	5
7. Práce v týmu.....	6
7.1 Rozdělení práce	6
7.2 Práce v týmu	6

Zdroje

Seznam příloh

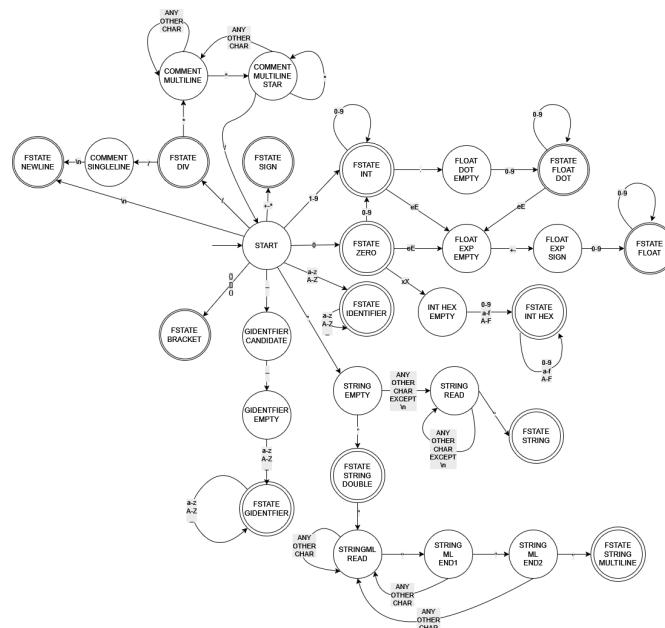
1. Úvod

Cílem projektu bylo navrhnout, implementovat a otestovat překladač jazyka IFJ25 (podmnožina jazyka WREN).

Účelem této dokumentace je přiblížit naši implementaci.

2. Lexikální analýza

Lexikální analýza byla jedna z prvních implementovaných komponent našeho překladače a je realizován za použití konečného automatu, který naleznete na obrázku 1. Vstupní funkcí do scanneru je funkce `getNextToken()`, která jako vstupní parametr bere ukazatel na int (chybový kód) a ukazatel na soubor (ze kterého následně čte). Dále obsahuje dva nepovinné ukazatele typu `size_t`, které umožňují sledovat pozici v standardním vstupu. Tuto funkci dále využívá syntaktická analýza (viz kapitola *Syntaktická a sémantická analýza*). Tato funkce pak vrací token, jehož podoba je definovaná strukturou `Token`, jehož obsahem je proměnná `TokenType`, která identifikuje typ token (např. `TOKEN_KEYWORD`, `TOKEN_IDENTIFIER`...).



Obrázek 1 – Konečný stavový automat

Konečný automat je implementován tak, aby se snažil vždy najít co nejdelší možnou lexému. Během tohoto zpracovávání může dojít k tomu, že skončí-li v koncovém stavu, může okamžitě začít zpracovávat další. S veškerými komentáři v zdrojovém kódu je naloženo jednoduchým způsobem – lexikální analyzátor je okamžitě zahodí a pokračuje k následující lexémě bez nutnosti externě volat znovu funkci `getNextToken()`.

Implementace se nachází v souborech:

- `lexikal_analyzator.c/h`
- `token.c/h`

3. Syntaktická a sémantická analýza

3.1 Syntaktická analýza

Syntaktická analýza využívá jak LL gramatiky, tak precedenční analýzy. Za užití rekurzivního sestupu dle LL gramatiky dochází k analýze tříd, funkcí/příkazů a řídicích struktur programu. V průběhu analýzy se opakovaně volá pro získání tokenu funkce `getNextToken()` (viz kapitola Lexikální analýza) a na základě tzv. Lookahead 1 (předvídání) se určí, které pravidlo bude použito. Každé pravidlo z LL gramatiky má funkci určenou k jeho kontrole.

V souladu se zadáním se nicméně LL gramatiky neužije pro zpracování výrazů. K tomu slouží precedenční analýza zdola nahoru. Tato část analýzy se spouští v rámci rekurzivního průchodu vždy, když se narazí na výraz. Ten je následně pomocí precedenční tabulky a zásobníku jednotlivých operátorů vyhodnocen.

Během vykonávání obou analýz dochází k sestavování abstraktního syntaktického stromu, se kterým dále pracuje jak sémantická analýza, tak generátor strojového kódu.

V případě detekce syntaktické chyby je tato detekována, analýza je ukončena a vrátí příslušný chybový kód definovaný v souboru `error_codes.c/h`.

3.2 Sémantická analýza

Po dokončení syntaktické analýzy je automaticky spuštěna analýza (na konci modulu `parserRun()` sémantická pomocí volání funkce `performSemanticAnalysis()`). Na svém vstupu akceptuje AST a globální tabulku symbolů. Následně je zajištěn rekurzivní průchod AST pomocí preorder postupu a udržuje si kontext aktuální třídy a lokální tabulky symbolů.

Po analýze každého uzlu AST se provede kontrola dle jeho typu. U výrazů s binárními operátory se kontroluje typová kompatibilita operandů a zároveň se ověřuje existence případných proměnných. Stejně tak se při volání funkcí kontroluje existence funkce a případně jestli dostává správný počet argumentů.

V případě detekce sémantické chyby, je analýza ukončena a vrací příslušný návratový kód, stejně jako v případě syntaktické a lexikální analýzy.

Implementace se nachází v souborech:

- `semantic_functions.c/h`
- `syntactic_analyzer.c/h`
- `prec_anal.c/h`

4. Generátor strojového kódu

Generátor strojového kódu je definovaný v souboru `machine_code_generator.c/h`, vstupním bodem do něj je funkce `generateIFJCode25()`, která je volána z centrální funkce `main()`. Jako vstupní parametr akceptuje ukazatel na soubor (standardní vstup). Zde je také volána funkce `parserRun()`. Tím jsou spuštěny veškeré analýzy (lexikální, syntaktická a sémantická) a na základě návratového kódu se rozhodne jestli se bude pokračovat v generování strojového kódu, nebo bude program neprodleně ukončen pro nalezenou chybu. V případě, že všechny analýzy proběhly úspěšně, generátor si vyžádá tabulku symbolů a syntaktický strom a zahájí generování.

Principem fungování generátoru je převod abstraktního syntaktického stromu na jednotlivé instrukce a jejich následný výpis na standardní výstup. Toto se děje rekursivním průchodem abstraktního syntaktického stromu (AST), kdy na základě něj dochází k zakládání návěští nových funkcí a případné skoky na ně, zajišťuje tvorbu a deklaraci proměnných a případně i jejich typovou kontrolu, ač tu se nepovedlo implementovat zcela 100%.

Pro zajištění použití unikátních návěští, názvů (pomocných) proměnných se používají k tomu určené globální čítače. Užití těchto globálních čítačů bylo výrazně jednodušší než předávání jejich hodnot mezi jednotlivými funkcemi, které s nimi pracují.

Případné ladící výstupy, stejně jako ve všech dalších částech programu, jsou vždy vkládány na standardní chybový výstup, aby nedocházelo k interferencím těchto informativních textů s užitečným kódem.

Implementace se nachází v souborech:

- `machine_code_generator.c/h`

5. Použité algoritmy a datové struktury

5.1 Tabulka symbolů

Tabulka symbolů je implementovaná pomocí výškově vyvážených binárních vyhledávacích stromů[1]. Tabulka symbolů je hierarchicky rozdělena na globální, třídní a lokální. Globální tabulka obsahuje všechny třídy a globální proměnné obsažené ve zdrojovém kódu. Každá třída má svou třídní tabulku, jež obsahuje všechny funkce (včetně jejich vstupních parametrů), gettery a settery v ní obsažené. Každá funkce má pak svou lokální tabulku, kde jsou uloženy všechny v ní definované (místní) proměnné.

Díky rozdělení tabulky symbolů na různé hierarchicky úrovně, bylo předejito velkým datovým strukturám, čímž došlo k elegantnějšímu členění dat a zároveň efektivnější práci s daty v nich (zejména rychlejší vyhledávání). Je samozřejmostí, že všechny úrovně tabulky symbolů sdílí jednotnou implementaci.

Kromě globální tabulky, která je pouze jedna, se všechny podružné tabulky na jedné úrovni shromažďují do zásobníků implementovaných pomocí jednosměrně vázaného seznamu. To zajišťuje hladký a jednoznačný průchod jednotlivých tabulek.

5.2 AST

Abstraktní syntaktický strom byl implementován pomocí n -árního nevyvažovaného stromu (vyvažování by vedlo k faktickému poškození stromu – proto byl také zavržen původní plán na implementaci pomocí shodné datové struktury jako tabulka symbolů). V praxi má v tomto stromě každý otec nejvýše 20 dětí (bezpečnostní okraj je 100 dětí na uzel).

Nevýhodou této implementace je poměrně vysoká náročnost na (re)alokaci paměti. Při každém přidání dítěte je volána funkce `realloc()`, která musí přkopírovat celé pole ukazatelů na nové místo, pokud původní blok nelze rozšířit. Pro uzel s n dětmi je tak provedeno n realokací a zkopírováno až $O(n^2)$ ukazatelů. Efektivnější by byla strategie geometrického růstu kapacity, která by snížila počet realokací na $O(\log n)$ a amortizovanou složitost přidání na $O(1)$.

6. Použité nástroje

Při vývoji našeho překladače jsme využívali běžně dostupné nástroje, které byly napříč týmem skoro totožné. Vývoj probíhal pod operačním systémem *Linux* nebo pod *WSL*. Jako editor a potažmo integrované vývojové prostředí (IDE) bylo využito *Visual Studio Code* s rozšířeními pro vývoj v jazyce C. Nad rámec těchto běžných rozšíření byl používán systém *GitLens* pro sledování toho, kdo psal jakou část kódu.

Pro správu verzí byl využit *Git*, *GitHub* a pro zjednodušení práce s *Git* byla využita grafická nadstavba *GitKraken*. Zároveň došlo k implementaci jednoduchého procesu kontinuální implementace a kontinuálního dodání (tzv. *CI/CD pipeline*), která při každém založení nového Pull-Request automaticky spustila doposud implementované testy.

Pro maximální standardizaci formátu zdrojových souborů byl nastaven jednotný editor config, jenž je přílohou této dokumentace.

7. Práce v týmu

7.1 Rozdělení práce

Práce na projektu byla v maximální možné míře rozdělena rovnoměrně a po předešlé dohodě mezi jednotlivými členy týmu. Při dělení práce byly zohledněny silné a slabé stránky jednotlivých členů týmu. Konkrétní rozdělení práce vizte v tabulce 1.

Jméno Příjmení	xlogin	Na čem pracoval	Podíl
Vojtěch Kadlec	xkadlev00	Generátor strojového kódu, symtable a další pomocné funkce.	25 %
Jakub Kosinka	xkosinj00	Lexikální a semantická analýza, generátor strojového kódu	25 %
Aleš Obr	xobrale00	Testy, dokumentace, semantická analýza, generátor strojového kódu, úpravy kódu, symtable, administrace	25 %
Tomáš Zvoniček	xzvoni00	Syntaktická analýza, LL gramatika, precedenční tabulka	25 %

Tabulka 1 – rozdělení práce

Vzhledem k tomu, že každá část měla svá úskalí a docházelo k vzájemné výpomoci na jednotlivých komponentech překladače, bylo v týmu odsouhlaseno rovnoměrné rozdělení bodů.

7.2 Práce v týmu

Práce v týmu byla rozhodnutím vedoucího a za souhlasu všech členů týmu rozdělena již v prvních týdnech výuky. Komunikace primárně probíhala přes *Discord* a popřípadě i v rámci repozitáře na *GitHub*. Taktéž proběhlo několik osobních setkání k ujasnění si některých záležitostí a utužení týmového ducha.

Obecně lze zhodnotit práci pozitivně. Během vývoje nenastaly závažnější týmové krize a ani se nestalo, že by některý člen odmítl spolupracovat.

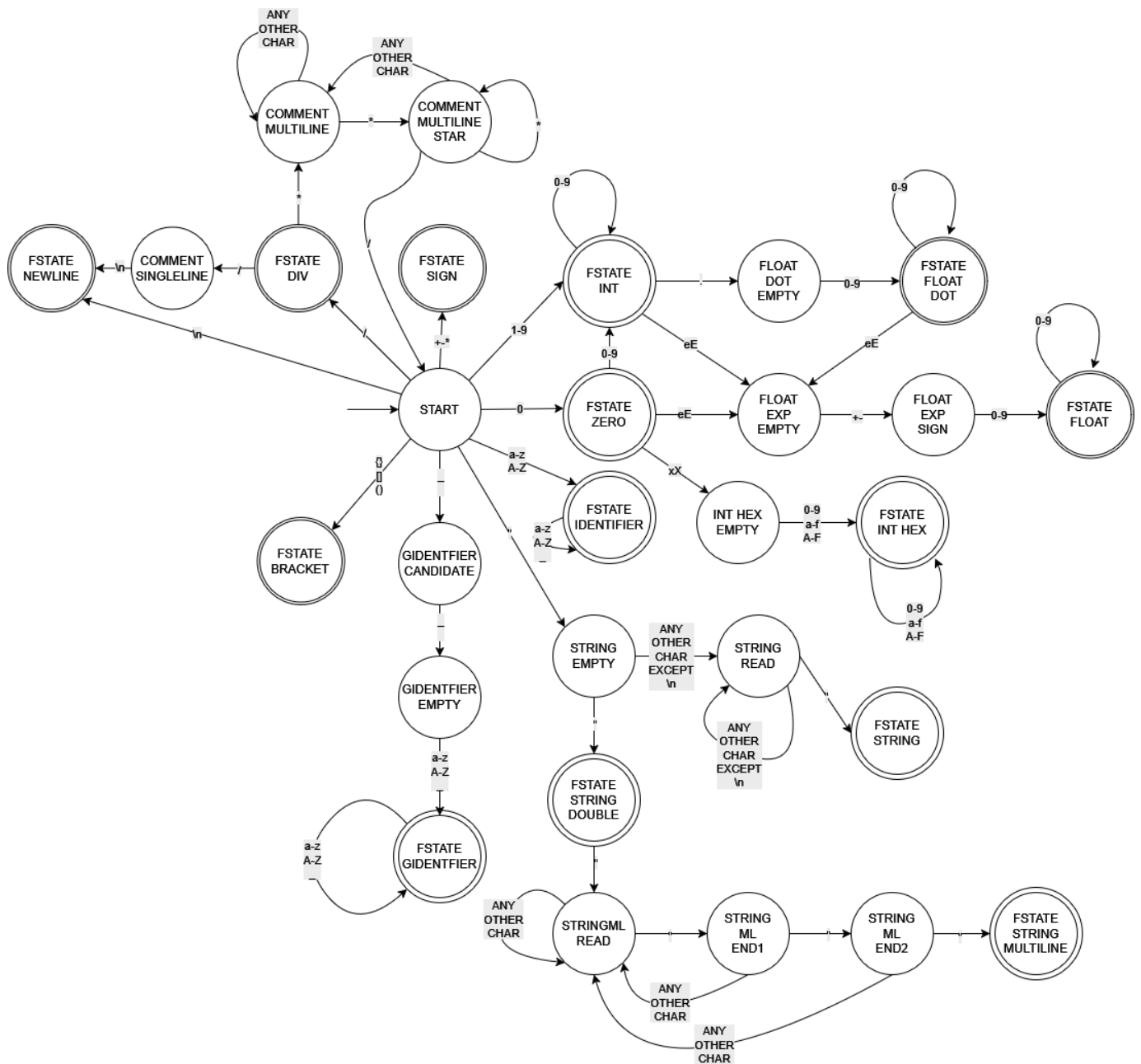
Zdroje

1 CORMÉN, Thomas H., LEISERSON, Charles E., RIVEST, Ronald L. a STEIN, Clifford. Introduction to Algorithms. 3. vyd. Cambridge, MA: MIT Press, 2009. ISBN 978-0262033848

Seznam příloh

1. Konečný automat lexikální analýzy	9
2. LL Gramatika	10
3. Precedenční tabulka	11
4. .editorconfig	12

1. Konečný automat lexikální analýzy



2. LL Gramatika

Non-terminal	import	class	static	var	id	{	}	(	)	,	=	if	while	return	int, str, null	NEWLINE	EOF
Program	1	1															1
ImportList	2 ϵ																ϵ
ImportStmt	3																
ClassList		4															ϵ
ClassDef		5															
ClassBody						6											
MemberList			7	7	7	ϵ											
Member			8,9,10,11	12	Error												
Params					13			ϵ									
ParamsTail*								ϵ	14								
Block						15											
StmtList				16	16	16 ϵ						16	16	16			
Stmt				17	18, 19	24						20	21	22, 23			
Args					25			25 ϵ							25		
ArgsTail*								ϵ	26								
Expression					PSA			PSA							PSA		

1. Program $\rightarrow$ ImportList ClassList
2. ImportList $\rightarrow$ ImportStmt ImportList $\mid \epsilon$
3. ImportStmt $\rightarrow$ import STRING for id
4. ClassList $\rightarrow$ ClassDef ClassList $\mid \epsilon$
5. ClassDef $\rightarrow$ class id ClassBody
6. ClassBody $\rightarrow$ { NEWLINE MemberList }
7. MemberList $\rightarrow$ Member MemberList $\mid \epsilon$
8. Member $\rightarrow$ static id (Params) Block
9. Member $\rightarrow$ static id Block

10. Member $\rightarrow$ static id = (Params) Block
11. Member $\rightarrow$ static var id
12. Member $\rightarrow$ var id
13. Params $\rightarrow$ id ParamsTail $\mid \epsilon$
14. ParamsTail $\rightarrow$, id ParamsTail $\mid \epsilon$
15. Block $\rightarrow$ { NEWLINE StmtList }
16. StmtList $\rightarrow$ Stmt StmtList $\mid \epsilon$
17. Stmt $\rightarrow$ var id
18. Stmt $\rightarrow$ id = Expression

19. Stmt $\rightarrow$ id (Args)
20. Stmt $\rightarrow$ if (Expression) Block else Block
21. Stmt $\rightarrow$ while (Expression) Block
22. Stmt $\rightarrow$ return Expression
23. Stmt $\rightarrow$ return
24. Stmt $\rightarrow$ Block
25. Args $\rightarrow$ Expression ArgsTail $\mid \epsilon$
26. ArgsTail $\rightarrow$, Expression ArgsTail $\mid \epsilon$
27. Expression $\rightarrow$ handled by precedence analysis (PSA)

3. Precedenční tabulka

Stack \ Input	+	-	*	/	<	<=	>	>=	==	!=	is	(	)	id	int	str	nil	\$END
+	>	>	<	<	>	>	>	>	>	>	>	<	>	<	<	<	<	>
-	>	>	<	<	>	>	>	>	>	>	>	<	>	<	<	<	<	>
*	>	>	>	>	>	>	>	>	>	>	>	<	>	<	<	<	<	>
/	>	>	>	>	>	>	>	>	>	>	>	<	>	<	<	<	<	>
<	<	<	<	<	>	>	>	>	>	>	>	<	>	<	<	<	<	>
<=	<	<	<	<	>	>	>	>	>	>	>	<	>	<	<	<	<	>
>	<	<	<	<	>	>	>	>	>	>	>	<	>	<	<	<	<	>
>=	<	<	<	<	>	>	>	>	>	>	>	<	>	<	<	<	<	>
==	<	<	<	<	<	<	<	<	>	>	>	<	>	<	<	<	<	>
!=	<	<	<	<	<	<	<	<	>	>	>	<	>	<	<	<	<	>
is	<	<	<	<	<	<	<	<	<	<	<	<	>	<	<	<	<	>
(	<	<	<	<	<	<	<	<	<	<	<	<	=	<	<	<	<	err
)	>	>	>	>	>	>	>	>	>	>	>	err	>	err	err	err	err	>
id	>	>	>	>	>	>	>	>	>	>	>	err	>	err	err	err	err	>
int	>	>	>	>	>	>	>	>	>	>	>	err	>	err	err	err	err	>
str	>	>	>	>	>	>	>	>	>	>	>	err	>	err	err	err	err	>
nil	>	>	>	>	>	>	>	>	>	>	>	err	>	err	err	err	err	>
\$END	<	<	<	<	<	<	<	<	<	<	<	<	err	<	<	<	<	err

4. .editorconfig

```
1  root = true
2
3  [*]
4  indent_style = tab
5  indent_size = tab
6  charset = utf-8
7  end_of_line = lf
8  insert_final_newline = true
9  trim_trailing_whitespace = true
10
11  [*.c]
12  # Specifické pro C soubory
13  indent_style = tab
14  indent_size = tab
15  max_line_length = 100
16
17  [*.h]
18  # Specifické pro hlavičkové soubory
19  indent_style = tab
20  indent_size = tab
21  max_line_length = 100
.
```